

Fast and Safe Trajectory Planning: Solving the Cobot Performance/Safety Trade-Off in Human-Robot Shared Environments

Alessandro Palleschi¹, Student Member, IEEE, Mazin Hamad², Student Member, IEEE, Saeed Abdolshah³, Manolo Garabini⁴, Sami Haddadin⁵, and Lucia Pallottino⁶, Senior Member, IEEE

Abstract—The rise of collaborative robotics has offered new opportunities for integrating automation into the factories, allowing robots and humans to work side-by-side. However, this close physical coexistence inevitably brings new constraints for ensuring safe human-robot cooperation. The current paramount challenge is integrating human safety constraints without compromising the robotic performance goals, which require minimization of the task execution time alongside ensuring its accomplishment. This letter proposes a novel robot trajectory planning algorithm to produce minimum-time yet safe motion plans along specified paths in shared workspaces with humans. To this end, a safety module was used to evaluate the safety of a time-optimal trajectory iteratively. A safe replanning module was developed to optimally adapt the generated trajectory online whenever the optimal plan violates dynamically provided safety limits. In order to preserve performance, a recovery trajectory planning algorithm was included such that the robot is allowed to restore higher speed motions as soon as the safety concern has been resolved. The proposed solution's effectiveness was evaluated both in simulations and real experiments with two robotic manipulators.

Index Terms—Robot safety, motion and path planning, human-centered robotics.

I. INTRODUCTION

RECENT years have seen an increased interest and effort in developing robotic automation solutions for new market areas, characterized by short product lifetimes and dynamic production changeovers. These requirements drive technology in the direction of scalable and flexible collaborative robots, mostly working in shared environments alongside humans [1].

Manuscript received December 21, 2020; accepted April 19, 2021. Date of publication April 30, 2021; date of current version May 13, 2021. This letter was recommended for publication by Associate Editor J. Zhang and Editor C. Gosselin upon evaluation of the reviewers' comments. This work was supported in part by the European Unions Horizon 2020 research and innovation program as part of the projects ILIAD under Grant 732737, and in part by the Italian Ministry of Education and Research in the framework of the CrossLab project (Departments of Excellence). (Corresponding author: Alessandro Palleschi.)

Alessandro Palleschi, Manolo Garabini, and Lucia Pallottino are with the Research Center "E. Piaggio", Department of Information Engineering, University of Pisa, 56126 Pisa, Italy (e-mail: alessandro.palleschi@phd.unipi.it; manolo.garabini@gmail.com; lucia.pallottino@unipi.it).

Mazin Hamad, Saeed Abdolshah, and Sami Haddadin are with the Munich School of Robotics and Machine Intelligence, Technical University of Munich (TUM), 80797 Munich, Germany (e-mail: mazinhamd@gmail.com; saeed.abdolshah@tum.de; sami.haddadin@tum.de).

This letter has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2021.3076968>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2021.3076968

Research on physical Human-Robot Interaction (pHRI) has also received growing attention recently [2], [3] to accelerate the employment of collaborative robots, especially in small and medium enterprises (SMEs). However, one of the key elements hindering full exploitation of these collaborative solutions is the difficulty of integrating human safety requirements with classical production constraints, e.g., high-speed operations, fast cycle time capabilities, and path constraints [4].

Typically, production constraints are addressed by planning path-constrained, time-optimal motions considering limits on actuator velocities, and accelerations using convex optimization techniques [5]. Such algorithms are mostly used as off-line batch methods for programming the robot to move at its full dynamic capabilities. However, these solutions cannot guarantee safety if the robot has to coexist with humans in unstructured, shared environments. Hence, for developing safe physical Human-Robot Collaboration (pHRC), beyond production constraints, it is of high importance to ensure human safety when robots operate in shared workspaces where collisions may happen.

Human safety requirements during pHRC led to the development of two main paradigms [6]: Speed and Separation Monitoring (SSM) [7], and Power and Force Limiting (PFL) [8]. In the SSM, the relative distance and velocity between the human and the robot are the main factors influencing the robot speed limit, which should guarantee a feasible complete stop before an impact can occur. On the other side, the PFL allows non-zero velocity contacts between the robot and the operator, as long as the energy transferred by the impact is below a certain safety threshold. PFL methods, such as, e.g., the Safe Motion Unit (SMU) scheme proposed by Haddadin *et al.* [9], limit the robot velocity to a safe level based on the robot dynamic properties (i.e., reflected inertia, the geometry of potential collision points and velocity) in addition to human injury database, reducing the traumatic effects of possible contacts.

From the performance point of view, the SSM paradigm may cause the robot to move unnecessarily slow or even stay still when it drives closer to the human worker, thus compromising productivity. Nevertheless, it allows the robot to move at full speed for high separating distances. In contrast, since it does not consider the human-robot separating distance, PFL could reduce the performance when the human is far away from the robot or the robot moves away from the operator [10]. However, it would allow higher speed motions w.r.t. SSM methods in case of close proximity. While these solutions can guarantee human safety, the development of a strategy able to ensure a decent level of performance for every possible situation is still an open problem. In this letter, we propose a novel trajectory planning

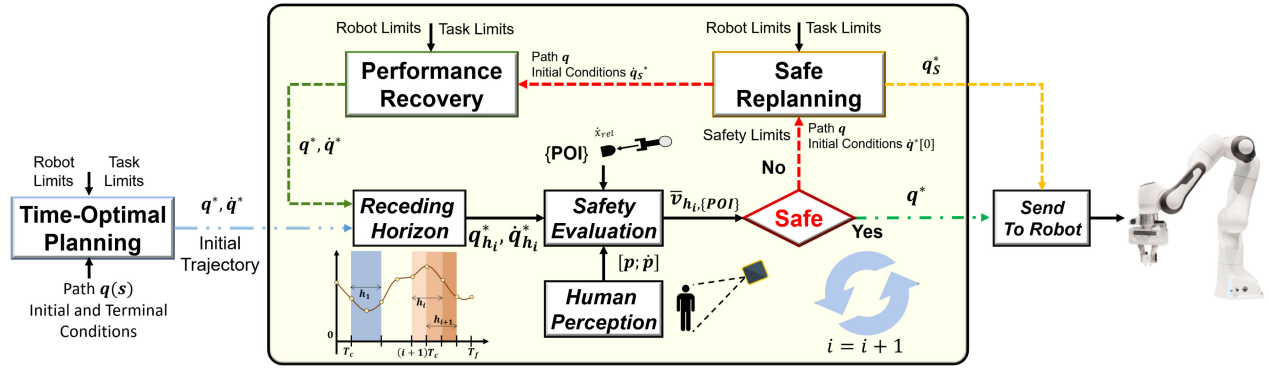


Fig. 1. Block diagram of the proposed discrete time fast and safe trajectory planning framework. At every cycle, based on the information about the human position and speed and the current plan, safety is possibly ensured by replanning the time evolution of the robot motion along the path.

algorithm capable of combining the benefits of both approaches to compute safe trajectories without compromising productivity.

The contributions of the letter are as follows. With the aim of maximizing the performance, we developed an optimization-based trajectory planning framework implemented in an iterative fashion to account for dynamic safety constraints generated online based on information on the human presence and motion provided by a perception system. A convex formulation for the path-constrained minimum-time trajectory planning problem with constraints on the robot velocities, accelerations, torques, and jerk is presented. This formulation considers jerk limits as hard constraints while preserving the convexity of the problem. The problem is solved to generate a reference minimum-time trajectory for the robot and replan online the temporal profile of the trajectory whenever the safety speed constraints are provided to the planner.

The proposed formulation:

- 1) is based on a general framework, applicable to generic paths, robots, and human trajectories;
- 2) adapts the temporal profile using a convex optimization algorithm, considering the whole dynamics of the robot and limits on the joints velocities, accelerations, and jerk;
- 3) includes biomechanics injury database and the actual robot's inertial properties into planning, not relying on conservative and simplified assumptions on robot motion and geometries;
- 4) dynamically evaluates and enforces online the safety constraints.

Furthermore, the proposed approach does not rely on reactive control strategies or path replanning, allowing the robot to safely adjust its motion while being consistent with the assigned path. Indeed, as highlighted in [4], it might not always be possible for the robot to modify its path, e.g., in manufacturing environments. With respect to classic velocity scaling solutions that solely rely on finding a scaling coefficient δ for the robot speed (see, e.g., [4], [10]), the proposed replanning approach based on convex optimization techniques can offer guarantees on the feasibility of the replanned trajectory, taking into account the robot actuation limits.

The outline of the letter is as follows: Section II presents the approach. Its effectiveness is evaluated through simulations and real experiments with collaborative robotic manipulators in a shared human-robot environment in Sections III and IV, respectively. Section IV also includes a comparison with a classic velocity scaling approach. Section V concludes the letter highlighting future research directions.

Algorithm 1: Safe Trajectory Planning.

Inputs: path q , horizon h , points of interest $\{\text{POI}\}$

- 1: $\{q^*, \dot{q}^*\} = \text{timeoptimalPlan}(q)$
- 2: $\text{SAFE} = \text{TRUE}$
- 3: **repeat**
- 4: $\{q_h^*, \dot{q}_h^*\} = \text{recHorizon}(q^*, \dot{q}^*, h)$
- 5: $\{\text{SAFE}, \bar{v}_h, \text{POI}\} = \text{safetyEval}(q_h^*, \dot{q}_h^*, \{\text{POI}\})$
- 6: **if** SAFE **then**
- 7: $\text{sendtoRobot}(q^*)$
- 8: **else**
- 9: $\{q_s^*, \dot{q}_s^*\} = \text{safeReplan}(q^*, \bar{v}_h, \text{POI}, \{\text{POI}\})$
- 10: $\text{sendtoRobot}(q_s^*)$
- 11: $\{q^*, \dot{q}^*\} = \text{perfRecover}(q_s^*)$
- 12: $h = \text{moveHorizon}()$
- 13: sleep until control period T_c has elapsed
- 14: **until** End of Task

II. PROPOSED SOLUTION

Our proposed planning framework has been designed to comply at least with the following conditions: i) the robot has no prior information about the current human position and his intended motion trajectory; ii) if it is not endangering any human, the robot should move at full speed; iii) the robot motion should be adapted to respect rising safety constraints due to the current environmental conditions, taking into accounts the actuation capabilities and without deviating from its path; iv) the robot should resume moving at its maximum allowed speed as soon as the safety concerns are resolved. Therefore, an iterative planning scheme was developed as described by Algorithm 1, while a block scheme highlighting its working principle is depicted in Fig. 1.

Given the desired path to be followed by the robot, first, an initial time-optimal trajectory, considering only the robot actuation limits and possible task-related constraints is generated (block *Time-Optimal Planning* in Fig. 1 and function **timeoptimalPlan** in Algorithm 1). After that, the safety of the generated time-optimal trajectory is assessed online by a *Safety Evaluation* module (block *SafetyEvaluation* in Fig. 1 and function **safetyEval** in Algorithm 1) that, based on the robot planned trajectory and the human data from a visual perception system, provides speed constraints for given point of interests (POIs) along the robot structure. If the current time-optimal trajectory violates some of these constraints, the time evolution of the path is re-planned online by solving a minimum-time path-constrained

problem that takes into account the safety constraints (block *SafeReplanning* in Fig. 1 and function **safeReplan** in Algorithm 1).

Finally, the new trajectory is sent to the robot, while a dedicated *Performance Recovery* module (function **perfRecover** in Algorithm 1) is used to generate a new time-optimal plan to be checked by the *Safety Evaluation* module, so as to restore higher speed motions as soon as the safety concerns are resolved.

In the following, a description of the design and working principles of the main blocks of our framework is presented.

A. Minimum-Time Trajectory Planning

The problem of generating optimal trajectories along a specified path can be efficiently tackled with state-of-the-art methods in case of limits on the joint velocities, accelerations, and torques. These methods translate the optimal control problem into a convex optimization problem [5].

However, in order to obtain smoother trajectories with less demanding actuator commands, we also include constraints on the jerk into the optimization problem.¹ The resulting optimization problem is described in the following.

Given a task and the related joint-space path $\mathbf{q} \in \mathbb{R}^n$, it can be typically represented by a function $\gamma(s) \in \mathbb{R}^n$, where s is a monotonically increasing scalar parameter $s(t) \in [0, 1]$, and n is equal to the number of joints of the robot (see e.g. [12]). If we want to compute the trajectory that minimizes the total time T needed for following the given path, it is possible to write the following optimization problem

$$\begin{aligned} & \min_{T, s(\cdot)} T \\ & \{s(0), \dot{s}(0), s(T), \dot{s}(T)\} = \{s_0, \dot{s}_0, s_T, \dot{s}_T\} \\ & \dot{s}(t) \geq 0 \\ & \{\dot{\mathbf{q}}(t), \ddot{\mathbf{q}}(s(t)), \ddot{\mathbf{q}}(s(t)), \boldsymbol{\tau}(s(t))\} \in \mathcal{Q}(s(t)) \\ & \text{for } t \in [0, T]. \end{aligned} \quad (1)$$

where $\mathcal{Q}(s(t))$ is the constraint set for the joint velocities, accelerations, jerks, and torques.² However, w.r.t. classical torque-level formulations, by introducing jerk constraints, the optimization problem is no longer convex [12].

Solving a non-convex problem is usually computationally demanding, and the time needed for finding the optimal solution makes it practically unfeasible to use this approach for online trajectory planning. On the other hand, solutions that try preserving the convexity by including jerk limits as penalty terms into the objective functions, similar to what is done in [5] for the torque rate, are able to reduce the jerk, but can not limit it to a specific value. Thus, they are not able to guarantee that the solution would not violate the jerk limits. For these reasons, here we present a different formulation for (1), which considers the jerk limits as hard constraints while still preserving the problem's convexity.

First, we reformulate Problem (1) defining the classic change of variables $b = \dot{s}^2$ and $a = b' = \partial b / \partial s$. Then, we use direct transcription to numerically solve it. Therefore, discretizing

s with $K + 1$ grid points s_k , the problem of optimizing the motion of a robot moving along the discretized joint path can be written as

$$\min_{a_k, b_k} \sum_{k=0}^{K-1} \frac{2\Delta s_k}{\sqrt{b_{k+1}} + \sqrt{b_k}} \quad (2a)$$

$$\text{s.t. } b_0 = 0 \text{ and } b_K = 0 \quad (2b)$$

$$(b_{k+1} - b_k) = 2a_k \Delta s_k \quad (2c)$$

$$(b_K - b_{K-1}) = 2a_K \Delta s_K \quad (2d)$$

$$0 \leq b_k \leq \bar{b}_k \quad (2e)$$

$$\underline{\mathbf{y}}_k \leq \mathbf{f}_k a_k + \mathbf{p}_k b_k \leq \bar{\mathbf{y}}_k \quad (2f)$$

$$\underline{\mathbf{y}}_K \leq \mathbf{f}_K a_K + \mathbf{p}_K b_K \leq \bar{\mathbf{y}}_K \quad (2g)$$

$$\underline{\ddot{\mathbf{q}}}_k \leq \ddot{\mathbf{q}}_k \leq \bar{\ddot{\mathbf{q}}}_k \quad (2h)$$

$$\text{for } k = 0, \dots, K-1, \quad (2i)$$

where we omit the dependency on s for the sake of clarity. We assumed b and a to be piecewise linear and piecewise constant, respectively, as in [5]. Additionally, $\mathbf{q}' = \partial \mathbf{q} / \partial s$, $\mathbf{q}'' = \partial^2 \mathbf{q} / \partial s^2$, $\Delta s_k = s_{k+1} - s_k$, $\bar{\mathbf{q}}$ and $\underline{\mathbf{q}}$ are the upper and lower bounds for the jerk, respectively, and

$$\begin{aligned} \mathbf{f}_k &= [\mathbf{m}_k^\top \mathbf{q}'_k^\top]^\top, \quad \mathbf{p}_k = [\mathbf{c}_k^\top \mathbf{q}''_k^\top]^\top, \\ \bar{\mathbf{y}}_k &= [(\bar{\boldsymbol{\tau}}_k - \mathbf{g}_k)^\top \bar{\ddot{\mathbf{q}}}_k^\top]^\top, \quad \underline{\mathbf{y}}_k = [(\underline{\boldsymbol{\tau}}_k - \mathbf{g}_k)^\top \underline{\ddot{\mathbf{q}}}_k^\top]^\top, \end{aligned}$$

where $\bar{\boldsymbol{\tau}}_k$, $\bar{\ddot{\mathbf{q}}}_k$, $\underline{\boldsymbol{\tau}}_k$, and $\underline{\ddot{\mathbf{q}}}_k$ are the upper and lower bounds for the joint torques and accelerations, respectively. For a formal definition of the vectors $\mathbf{m}$, $\mathbf{c}$, and $\mathbf{g}$ the interested reader can refer to [5]. From (2), the jerk $\ddot{\mathbf{q}}_k$ can be written using finite difference approximation

$$\ddot{\mathbf{q}}_k \approx \frac{\ddot{\mathbf{q}}_{k+1} - \ddot{\mathbf{q}}_k}{\Delta t_k} = (\sqrt{b_{k+1}} + \sqrt{b_k}) \frac{\ddot{\mathbf{q}}_{k+1} - \ddot{\mathbf{q}}_k}{2\Delta s_k}. \quad (3)$$

As expected, using this formulation constraints (2h) are non-convex. However, it is possible to substitute $\ddot{\mathbf{q}}_k$ with the following upper bound

$$\ddot{\mathbf{q}}_{ws,k} = \left(\sqrt{b_{k+1}} + \sqrt{b_k} \right) \frac{\ddot{\mathbf{q}}_{k+1} - \ddot{\mathbf{q}}_k}{2\Delta s_k} \quad (4)$$

where we replaced b_k and b_{k+1} with their upper bounds. In practice, we substituted Δt_k with its minimum value, constraining the worst-case jerk. Therefore, using this convex approximation for the constraints (2h), we obtain the convex problem

$$\min_{a_k, b_k} \sum_{k=0}^{K-1} \frac{2\Delta s_k}{\sqrt{b_{k+1}} + \sqrt{b_k}} \quad (5a)$$

$$\text{s.t. } (2b) - (2g) \quad (5b)$$

$$\underline{\ddot{\mathbf{q}}}_k \leq \left(\sqrt{b_{k+1}} + \sqrt{b_k} \right) \frac{\ddot{\mathbf{q}}_{k+1} - \ddot{\mathbf{q}}_k}{2\Delta s_k} \leq \bar{\ddot{\mathbf{q}}}_k \quad (5c)$$

$$\text{for } k = 0, \dots, K-1. \quad (5d)$$

Note that the solution of Problem (5) is still feasible for the original problem (1), since its feasible set is a subset of that of (1). More details about the computational advantages and the

¹ Smooth trajectories with reduced jerk also have the nice feature of increasing the operator's acceptance of pHRC [11]

² The assigned joint space path can be obtained from the Cartesian path of the end-effector using any classical inverse kinematics algorithm. Therefore, any redundancy of the collaborative robot is not exploited while planning the time-optimal motion, but possibly during the inverse kinematics stage.

optimality of the solution for this convex reformulation of the problem are reported in the Appendix.

B. Safety Evaluation

For the safety evaluation, we exploit a unified safety framework that relies on biomechanical injury information to ensure the human coworker safety in case of collision, the safe motion unit (SMU) [9]. The considered interaction/collision model, following the approach introduced in [9] and further elaborated in [13], focuses on capturing the instantaneous contact dynamics between two colliding objects. Using a scalar mass and its velocity (together with the curvature at impact/contact location), the energy exchange between the robot (possibly with additional payload) and an impacted human can be summarized in terms of the reflected robot dynamics along the chosen motion direction [14].

The SMU provides the velocity limits which guarantee that the traumatic effects of every possible contact phase may not exceed a certain level of minor injury (e. g. a bruise, contusion) or, more strictly, even no injury at all [9]. As for the case of robots with sharp tools, integrating effective physical collision detection and reaction schemes [15], it was proved in [16] that equipping robot holding a knife with a sensitive collision detection and effective stopping strategy can protect the human from incised wounds at velocities up to 0.75 m/s. The maximum braking distance for a robot handling sharp tools could be set using penetration depth as a severity index for possible human contacts. Accordingly, velocity limits for such tasks can also be imposed and integrated under the SMU framework [16].

To ensure human safety by robot velocity control, the robot reflected mass is evaluated in certain motion direction at a number of points of interest (POIs) along the robot structure. The curvature at these possible impact locations is encoded together with biomechanical injury information against the robot reflected mass and Cartesian velocity. Given the reflected mass in the current robot configuration and along its motion direction, a maximum safe relative speed can be evaluated from the corresponding safety curve. This upper relative speed limit can be calculated from a simple linear regression relationship

$$v_{\max}(m) = \text{reg. lim} [c_1(i, \mathbf{p}_i) m + c_2(i, \mathbf{p}_i), v_1, v_2], \quad (6)$$

with $c_1(i, \mathbf{p}_i) < 0$ and $c_2(i, \mathbf{p}_i) > 0$ being the coefficients of the delimiting safety/injury curves for contact surface primitive i , m the reflected mass, and v_1, v_2 denoting the limits for cut-off minimum and maximum velocity. The contact primitive captures the geometrical information at the impact location, which is a key factor regarding how much energy is transferred to the impacted human tissue and the possible resulting injury. The complete detailed SMU algorithm can be found in [9], while a diagram summarizing the working principles of the SMU is reported in Fig. 2. Some background information on the dynamic model used to characterize the collisions/interactions and the computation of the reflected mass are reported in the Appendix.

The SMU is the core of our **Safety Evaluation** module, implemented in a receding horizon fashion. The working principle of this module is described by Algorithm 2. Given the list of robot POIs $\{\mathbf{POI}\}$, at every control cycle, the SMU is used to evaluate the safety of the planned trajectory over a user-defined monitoring time-horizon. By evaluating the safety of only a part of the trajectory, we reduce the computational burden of performing at each time step the evaluation for the complete

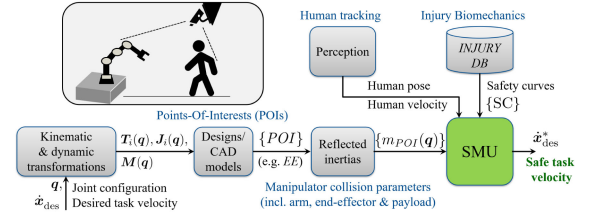


Fig. 2. The Safe Motion Unit (SMU) pipeline. This injury biomechanics-based approach ensures human safety for robot manipulators during pHRI.

Algorithm 2: Safety Evaluation Module.

```

safetyEval( $q_h^*, \dot{q}_h^*, \{\mathbf{POI}\}$ )
1: SAFE = TRUE
2:  $\bar{v}_{\mathbf{POI}} = \emptyset$ 
3:  $\{p, \dot{p}\} = \text{getPersonData}()$ 
4: if  $p \neq \emptyset$  then
5:   return SAFE,  $\bar{v}_{\mathbf{POI}}$ 
6: for  $\mathbf{POI} \in \{\mathbf{POI}\}$  do
7:   for  $\{q, \dot{q}\} \in \{q_h^*, \dot{q}_h^*\}$  do
8:      $d = \text{getDistance}(p, \mathbf{POI})$ 
9:     if  $d \leq \theta$  then
10:       $v_{\mathbf{POI}} = \text{getPOIVelocity}(q, \dot{q}, \mathbf{POI})$ 
11:       $m = \text{getReflectedMass}(q, \mathbf{POI}, p, \dot{p})$ 
12:       $v_{\max} = \text{SMU}(m)$ 
13:       $\bar{v}_{\mathbf{POI}}.\text{insert}(v_{\max})$ 
14:      if  $v_{\max} < |v_{\mathbf{POI}}|$  then
15:        SAFE = FALSE
16:       $\bar{v}_{\{\mathbf{POI}\}}.\text{insert}(\bar{v}_{\mathbf{POI}})$ 
17: return SAFE,  $\bar{v}_{\{\mathbf{POI}\}}$ 

```

path (even for points distant in time), given the current position of the person. On the other hand, deriving the safety constraints for the trajectory over a horizon instead of just the next point to send might possibly allow the robot to be more reactive, adapting its trajectory more smoothly and avoiding sudden (and possibly unfeasible) decelerations.

At each time step, the safety module receives the next joint positions and velocities of the trajectory within a horizon h . Then, it acquires information on the human presence from the perception system (function **getPersonData** in Algorithm 2). If no human is detected, the trajectory over the horizon is labelled as **SAFE**. If a person is detected, instead, it computes the maximum speed $v_{\max}$ for each trajectory point over the monitoring horizon and for each robot POI contained into the POI list $\{\mathbf{POI}\}$, using (6). To reduce the computational burden and avoid risks of unnecessary performance reductions, we introduced a safety threshold for the relative robot POI-human distance d . Indeed, if the person is not relatively close to the robot POI, it is not necessary to activate the SMU, and the maximum speed is set to the actual robot POI speed $|v_{\mathbf{POI}}|$. These safe values are then compared with the actual POI speed and, if for at least one robot POI and at least one trajectory point the safe value is below the actual speed, the planned trajectory is labelled as not safe, and sent to the **Safe Replanning** module for generating a new plan compliant with the new safety limits.

C. Time-Optimal Safe Replanning

The output of our safety module is a vector containing the maximum velocities for the robot POIs, computed on the given

horizon h . In case the trajectory is not safe, it is necessary to adapt it to be compliant with the new limits. We designed a safe replanning module that, based on the same convex optimization approach presented in Section II-A, computes an optimal trajectory compliant with the safety limits, the robot actuation capabilities, and the task-related constraints. The trajectory is replanned using the following modified version of Problem (5)

$$\min_{a_k, b_k} \sum_{k=0}^{L-1} \frac{2\Delta s_k}{\sqrt{b_{k+1}} + \sqrt{b_k}} \quad (7a)$$

$$\text{s.t. } b_0 = \dot{s}_0^2 \text{ and } b_L = 0, \quad (7b)$$

$$(2c) - (2d), \quad (7c)$$

$$0 \leq b_k \leq \bar{b}_S(s_k), \quad (7d)$$

$$(2f) - (2h), \quad (7e)$$

$$\text{for } k = 0, \dots, L-1, \quad (7f)$$

where L is the length of the remaining part of the path $\mathbf{q}$, $\dot{s}_0$ is the current initial condition for the robot path velocity, and $\bar{b}_S$ are the safe-consistent speed constraints. The advantage of this formulation, w.r.t. less computational demanding velocity scaling techniques, is that the safe replanning is guaranteed to generate a feasible and safe trajectory compliant with the robot actuation capabilities and the task constraints.

It is worth noting that Problem (7) with the safety constraints is still convex, since the speed constraints are embedded into the upper bounds on the variable b . For each value in $\bar{\mathbf{v}}_{h, \text{POI}}$, the constraints can be expressed as

$$\begin{aligned} |\mathbf{J}_v \dot{\mathbf{q}}| \leq |\bar{\mathbf{v}}_{h, \text{POI}}| &\leftrightarrow (\mathbf{J}_v \dot{\mathbf{q}})^2 \leq (\bar{\mathbf{v}}_{h, \text{POI}})^2 \leftrightarrow \\ &\leftrightarrow (\mathbf{J}_v \mathbf{q}')^2 b \leq (\bar{\mathbf{v}}_{h, \text{POI}})^2 \leftrightarrow b \leq \frac{(\bar{\mathbf{v}}_{h, \text{POI}})^2}{(\mathbf{J}_v \mathbf{q}')^2}, \end{aligned} \quad (8)$$

where $\mathbf{J}_v$ is the Jacobian matrix for linear motions associated with the robot POI, and where we exploited the equality $\dot{\mathbf{q}} = \mathbf{q}' \sqrt{b}$. Thus, the constraints on $\mathbf{v}_{h, \text{POI}}$ can be rewritten into the form $b \leq \bar{b}_S$, directly acting as upper bounds on b . Even for the case of multiple robot POIs, it is sufficient to consider only the most restrictive upper bound for each s over the monitoring horizon $\bar{b}_S(s) = \min\{\bar{b}(s), \min_{\{\text{POI}\}} (\frac{\bar{\mathbf{v}}_{h, \text{POI}}}{\mathbf{J}_v \mathbf{q}'(s)})^2\}$, while $\bar{b}_S(s) = \bar{b}(s)$ for the points outside the horizon.

D. Performance Recovery

Once the new plan has been obtained, the next point of the trajectory $\mathbf{q}_S^*$ is sent to the robot and the monitoring window is shifted ahead to the next point. However, before providing the **Safety Evaluation** module the next horizon, a **Performance Recovery** procedure is performed to compute a new optimal trajectory that does not consider safety speed constraints. This is achieved by solving an optimization problem with the same structure as Problem (5), optimizing over the path $\mathbf{q}_S^*$, with consistent initial conditions. The new optimal trajectory is then provided as the input of the **Safety Module**. The aim of this recovery procedure is to give to the **Safety Evaluation** module the fastest dynamically consistent trajectory at every cycle. Therefore, we are able to restore high-speed motions as soon as the safety issue that triggered the safe replanning has been

resolved, or to evaluate the safety limits over the fastest dynamically possible trajectory. We will show with simulations that, without the performance recovery, the planned trajectory might turn out to be unnecessarily conservative and the performance of the robotic system can be enhanced using the recovery action.

E. Considerations on Replanning and Path Constraints

The proposed method allows regulating the motion of the robot while maintaining path-consistency since it is not always possible to arbitrarily relax or modify the path without violating some of the production constraints [4]. In addition, limiting the robot to follow a fixed path can increase the predictability and legibility of the motion, with beneficial effects on safety and comfort perceived by the human operator [17]. However, it might reduce performance in specific conditions, e.g., a human operator standing along the path. The robot would reduce its speed as long as it moves toward the human. With our framework, the robot would let the human resolve the conflict by, e.g., moving away. However, a more proactive policy might be possible, with the robot replanning its path in the case of prolonged idle times. Nevertheless, such a proactive approach should be carefully designed to generate acceptable, efficient, and predictable motions and paths.

III. SIMULATIONS

In this section, the effectiveness of our proposed framework is assessed through simulations. The robot employed is a Franka Emika Panda manipulator equipped with a Pisa/IIT SoftHand as end-effector. The task assigned to the robot is to follow a circular trajectory. We simulate the presence of a human operator in the robot workspace, following an assigned trajectory $\mathbf{p}(t)$. For the simulated case, only one robot POI, placed at the center of the SoftHand, is considered, while the safety curves used by the SMU encode exemplary injury/safety information of collision incidents against the human chest. The robot motion along the specified circular path is planned using four different approaches, whose performance in terms of safety and trajectory times are then compared and discussed. For all the experiments, the simulated human worker follows the same trajectory $\mathbf{p}(t)$. Note that the trajectory followed by the human is not known in advanced by the robot, but we assume that a perception system can provide the actual human data at every cycle. For the limits on the robot velocity, acceleration, and jerk, we used the nominal values provided by Franka.³

In the first scenario, named **Time-Optimal**, the motion of the robot is planned to fully exploit its actuation capabilities, solving the minimum-time optimization problem (5a) presented in Sec. II-A. The safety module evaluates the safety of the optimal plan, neglecting the online safe-replanning. The proposed scenario has been chosen to highlight as the time-optimal trajectory can violate the safety constraints.

The second method, named **Conservative**, solves the same minimum-time optimization problem as in the time-optimal approach, but the limits on the joint velocities have been scaled to the point at which the planned trajectory is considered safe by the safety module. This scenario is instrumental for comparison of a safe but conservative trajectory.

The third method, named **Replanning without Recovery**, uses our proposed approach, but without using the performance

³https://frankaemika.github.io/docs/control_parameters.html

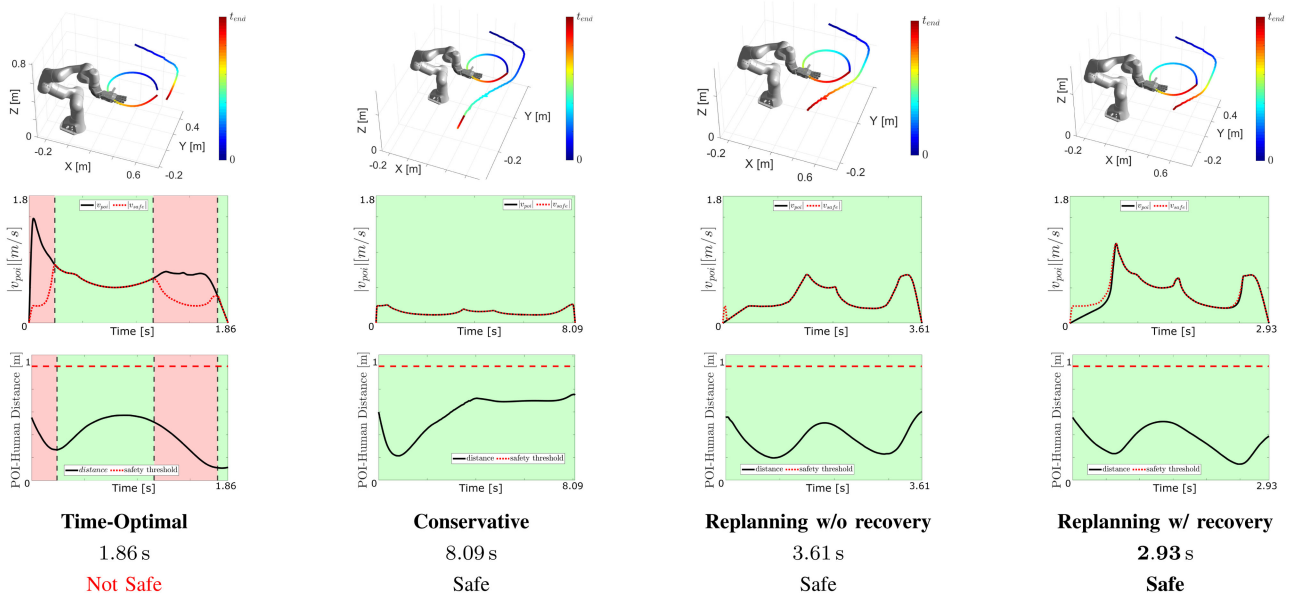


Fig. 3. For each column, from top to bottom: visualization of the trajectory of both the POI and the person; plot of the planned module of the POI speed, solid line, and the corresponding safety limit computed by the SMU, dashed line (red area corresponds to violated safety constraints); time evolution of the relative POI-Human distance, solid line, and the corresponding safety threshold, dashed line.

recovery step. In contrast, the fourth method, named **Replanning with Recovery**, uses our proposed approach with the inclusion of the performance recovery step.

For the third and fourth methods, the planning loop runs at 25 Hz, the monitoring horizon length has been set to 0.32 s, and the safety threshold for the robot POI-human distance has been set to 1 m. For all the four cases, the optimization problems presented in Sec. II have been implemented in C++ using CasADI [18] and the NLPs have been solved by the interior point solver IPOPT [19] with ma57-HSL linear solver.⁴ The results of the simulations for the four scenarios, in terms of trajectory, speed of the robot POI, safe speed computed by the SMU, and relative robot POI-human distance are reported in Fig. 3. The total trajectory time and the safety of the planned trajectories as evaluated by the **Safety Module** are also shown. We recall that a plan is considered to be **unsafe** if, for at least one trajectory point, the speed of the robot POI is greater than the safe speed maximum limit.

As expected, the motion planned using the Time-Optimal approach is the fastest but turns out to be not safe since we do not consider the human presence. Indeed, there are two intervals, highlighted in red in the first column of Fig. 3, where the planned robot POI speed is greater than the safe one. It is worth noting that, even if the relative robot POI-human distance is always below the 1 m safety threshold, there are intervals where the time-optimal trajectory is still safe (highlighted in green), due to the non-conflicting direction of motion between the robot and the human. On the other hand, the motion planned using the Conservative approach is able to ensure safety but at the expense of a larger total trajectory time (see second column of Fig. 3). Indeed, to obtain a safe trajectory without using any online replanning strategy, we needed to scale speed limits by six times. Our approach, instead, can overcome those issues,

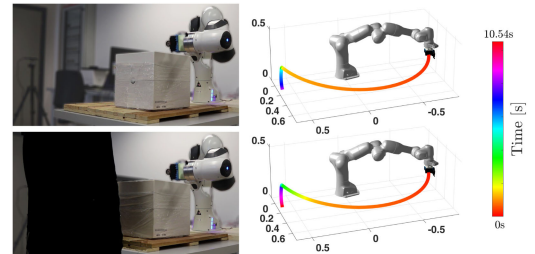


Fig. 4. Trajectory without (top) and with (bottom) the human operator.

thanks to the safe replanning strategy implemented. With and without the recovery action, the total time is higher than the one obtained with the Time-Optimal strategy, but the trajectory is considerably faster than the one provided by the Conservative approach (see third and fourth column of Fig. 3). It is worth noting as the presence of the performance recovery stage allows the robot to increase the performance further, reducing the time needed to complete the task by about 18%, while ensuring safety. Clearly, the computational effort increases since we need to solve two optimization problems at every cycle, and the safe replanning is expected to be activated more often.

IV. EXPERIMENTAL VALIDATION

Unwrapping Task. Our approach has also been evaluated experimentally on a scenario relevant to intralogistics, i.e., an autonomous single-object unwrapping task.

We used a fixed-base autonomous unwrapping robot, composed of a Franka Emika Panda arm with a custom-designed cutting end-effector: a concealed round actuated blade presented in [20]. The robot has to perform an unwrapping task on a single object brought to the cutting station by a human operator. First, the robot moves from the homing position, see Fig. 4, near the object to unwrap. The second phase is the actual cutting action,

⁴“HSL. A collection of Fortran codes for large scale scientific computation. <http://www.hsl.rl.ac.uk/>”

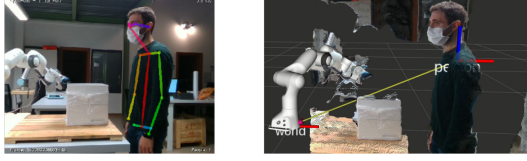


Fig. 5. Example of human body keypoints detection using OpenPose and corresponding 3D position of the chest.

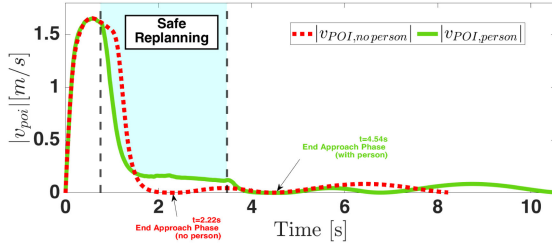


Fig. 6. Speed profile for the robot POI when the person is present (green solid line) and when is not present (red dashed line). The area highlighted represents the interval where the safe replanning procedure is performed.

TABLE I
DURATION OF THE TRAJECTORY AND THE TWO PHASES OF THE TASK

	Total Time	Approach Time	Cutting Time
Without Person	8.22 s	2.22 s	6.00 s
With Person	10.54 s	4.54 s	6.00 s

where the robot moves slowly to avoid damaging the object and ensuring the film engagement.

We tested two cases: one with a human operator standing near the object, the other without close proximity. We considered only one robot POI, on the cutting end-effector, and the safety curves used by the SMU are again related to exemplary collision outcomes with the human chest. The human perception model used for the experiment is based on OpenPose [21], a real-time multi-person visual perception software to detect human body, hand, facial, and foot keypoints. We used OpenPose together with a depth camera, an Intel Realsense D415, to extract the coordinates of the human keypoints and then of the chest.⁵ An example of the human keypoints detected by OpenPose and the extracted 3D position of the chest is reported in Fig. 5. The planning algorithm runs at 25 Hz, with a monitoring horizon of 0.12 s and a safety threshold of 0.6 m.

In Fig. 6 are reported the speed profiles for the robot POI. Table I reports the time for the whole trajectory and the two task phases. As shown in Fig. 6 (area highlighted in cyan), the presence of the human operator triggers the activation of the safe replanning, reducing the robot POI speed during the approach phase. On the other hand, if there is no person our planning method allows maximizing the performance, thanks to the time-optimal planning module used to generate the initial plan.

Comparison with a velocity scaling approach. We compared our method with a velocity scaling approach (VS in the

⁵OpenPose might allow retrieving the position of body parts other than the chest. Provided that suitable injury data for different body parts are available, the SMU could compute the safe speed based on the closest body part to the robot.

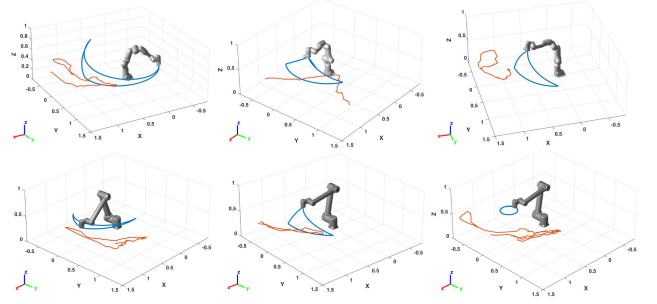


Fig. 7. Examples of trajectories and human motions.

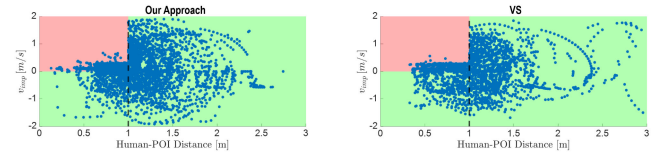


Fig. 8. Impact velocities and human-POI distances. Left: our approach; right: VS.

following). In VS, the time-optimal replanning procedure presented in Sec. II-C is replaced by a module that, at each iteration, given the planned velocity $\dot{\mathbf{q}}$ and the safe maximum velocities for the POIs returned by the SMU, generates a scaled velocity $\dot{\mathbf{q}}_S = \delta \dot{\mathbf{q}}$, $\delta \in [0, 1]$. The scaling factor is computed as $\delta = \min_{\text{POI}} \|\bar{\mathbf{v}}_{\text{POI}}\| / \|\mathbf{J}_v \dot{\mathbf{q}}\|$, where $\mathbf{J}_v$ is the Jacobian matrix for linear motions associated with the robot POI. We performed 80 tests (8 trajectories, 5 tests each for the two methods for each trajectory) with a Franka Emika Panda arm and a Universal Robot UR10e manipulator. Figure 7 shows examples of the trajectories and human motions recorded during the experiments. We considered one robot POI at the end of the last link and safety curves related to collision outcomes with the human chest. The human perception, based on OpenPose, runs at 15 Hz and the planning loop runs at 40 Hz with a monitoring horizon of 25 ms. The safety threshold is set to 1 m. The following table summarizes the results, reporting the percentage of completed tests, the average and the standard deviation of the human-POI distance, and the average velocity of the POI along the impact direction (only positive values):

	Completed	$\bar{d}_{\text{HPOI}}$	$\sigma_{d_{\text{HPOI}}}$	$\bar{v}_{\text{imp}}$
Our Approach	100%	1.12 m	0.38 m	0.47 m/s
VS	82.5%	1.08 m	0.43 m	0.41 m/s

It can be noted as, whilst our approach succeeded in completing all trajectories, using VS resulted in failures in some cases. These failures are due to unfeasible decelerations commanded to the robot when simply scaling the velocity to the safe value. In fact, VS does not consider limits actuation capabilities. Hence, unlike our solution, it can not guarantee the feasibility of the scaled trajectory. Figure 8 shows the impact velocities and the relative human-POI distances registered during the experiments. Both approaches allow reducing the POI speed when the relative distance is less than the safety threshold and the robot is approaching the human (red area in Fig. 8). In all other cases, instead, higher speeds are allowed since they are not a concern for safety. Examples of the behavior of the robots using the two

methods with different human motions are collected into the video available in the attached multimedia material.

V. CONCLUSION

In this letter, we presented a trajectory planning algorithm to plan fast and safe motions for collaborative robots in shared environments. The algorithm is based on an iterative procedure that can ensure the nearby human safety by replanning online the time evolution of the motion of the robot along the path based on the human data. The simulations and experimental validation have shown the effectiveness of the approach in producing safe but still efficient trajectories for the robot. Future works will focus on testing the performance in case multiple robot POIs and human body parts are used by the SMU when computing the safety constraints.

APPENDIX

Convex jerk-limited planning. We performed simulations solving both Problem (2) and (5) for a set of random trajectories using a Franka Emika Panda arm. Each trajectory has been discretized using 200 grid points for the path variable s . The problems have been implemented in Matlab using CasADI with the ma57-HSL linear solver for IPOPT, and with the same initial conditions. The following table reports statistics on the solutions to the problems and the solving times

$\tilde{q}_{NC}$	$\tilde{q}_C$	$\Delta \bar{t}_{opt}$	$\bar{t}_{IPOPT}^{NC}$	$\bar{t}_{IPOPT}^C$
1.00	0.99	5.72%	1128.4 ms	82.23 ms

The superscripts NC and C denote the results for problem (2) and (5), respectively, $\tilde{q}$ is the normalized peak for the jerk signal, i.e., $\tilde{q} = \max_i \{|\ddot{q}_i[t_k]|/\tilde{q}_i\}$, and $\tilde{q}$ represents the average over the different trajectories of the normalized peak for the jerk. The values $\bar{t}_{IPOPT}^{NC}$, $\bar{t}_{IPOPT}^C$, are the average time for IPOPT to solve Problem (2) and (5), respectively, and $\Delta \bar{t}_{opt}$ is the average difference between the optimal times, computed as $t_{opt}^C - t_{opt}^{NC}$. The solutions for (5) are suboptimal, but the problem is much faster to be solved, with an average time-reduction of around 93%.

Reflected mass computation. The rigid robot dynamic model can be expressed as $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau + \tau_{ext}$, where $q \in \mathbb{R}^n$ are the generalized link coordinates, $M(q) \in \mathbb{R}^{n \times n}$ is the symmetric, positive definite mass matrix, $C(q, \dot{q}) \in \mathbb{R}^{n \times n}$ is the Coriolis and centrifugal matrix, and $g(q) \in \mathbb{R}^n$ is the gravity torque vector [22]. The joint torques and the external torques are denoted by $\tau \in \mathbb{R}^n$ and $\tau_{ext} \in \mathbb{R}^n$, respectively. The so-called reflected mass is the scalar mass perceived at the manipulator end-effector along certain motion direction during an impact with the robot [23]. It can be calculated in a normalized Cartesian direction $u \in \mathbb{R}^3$ as $m_u(q) = [u^T \Lambda_v^{-1}(q)u]^{-1}$, where $\Lambda_v^{-1}(q)$ is the translational pseudo kinetic energy matrix that can be extracted from the partitioned inverse of the kinetic energy matrix [23].

REFERENCES

- [1] A. Cherubini *et al.*, "Collaborative manufacturing with physical human-robot interaction," *Robot. Comput. Integr. Manuf.*, vol. 40, pp. 1–13, 2016.
- [2] P. Tsarouchi *et al.*, "Human-robot interaction review and challenges on task planning and programming," *Int. J. Comput. Integr. Manuf.*, vol. 29, no. 8, pp. 916–931, 2016.
- [3] A. Ajoudani *et al.*, "Progress and prospects of the human-robot collaboration," *Auton. Robots*, vol. 42, no. 5, pp. 957–975, 2018.
- [4] A. M. Zanchettin, N. M. Ceriani, P. Rocco, H. Ding and B. Matthias, "Safety in human-robot collaborative manufacturing environments: Metrics and control," *IEEE Trans. Autom. Sci. Eng.*, vol. 13, no. 2, pp. 882–893, Apr. 2016.
- [5] D. Verscheure, B. Demeulenaere, J. Swevers, J. De Schutter, and M. Diehl, "Time-optimal path tracking for robots: A convex optimization approach," *IEEE Trans. Automat. Contr.*, vol. 54, no. 10, pp. 2318–2327, Oct. 2009.
- [6] V. Villani *et al.*, "Survey on human-robot collaboration in industrial settings: Safety, intuitive interfaces and applications," *Mechatronics*, vol. 55, pp. 248–266, 2018.
- [7] J. A. Marvel *et al.*, "Implementing speed and separation monitoring in collaborative robot workcells," *Robot. Comput. Integr. Manuf.*, vol. 44, pp. 144–155, 2017.
- [8] P. Aivaliotis *et al.*, "Power and force limiting on industrial robots for human-robot collaboration," *Robot. Comput. Integr. Manuf.*, vol. 59, pp. 346–360, 2019.
- [9] S. Haddadin *et al.*, "On making robots understand safety: Embedding injury knowledge into control," *Int. J. Rob. Res.*, vol. 31, no. 13, pp. 1578–1602, 2012.
- [10] N. Lucci, B. Lacevic, A. M. Zanchettin, and P. Rocco, "Combining speed and separation monitoring with power and force limiting for safe collaborative robotics applications," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 6121–6128, Oct. 2020.
- [11] P. A. Lasota *et al.*, "Analyzing the effects of human-aware motion planning on close-proximity human-robot collaboration," *Hum. Factors*, vol. 57, no. 1, pp. 21–33, 2015.
- [12] A. Palleschi, M. Garabini, D. Caporale, and L. Pallottino, "Time-optimal path tracking for jerk controlled robots," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 3932–3939, Oct. 2019.
- [13] N. Mansfeld, M. Hamad, M. Becker, A. G. Marin, and S. Haddadin, "Safety map: A unified representation for biomechanics impact data and robot instantaneous dynamic properties," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1880–1887, Jul. 2018.
- [14] M. Hamad, N. Mansfeld, S. Abdolshah, and S. Haddadin, "The role of robot payload in the safety map framework," in *IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 195–200.
- [15] S. Haddadin, A. Albu-Schaffer, A. De Luca, and G. Hirzinger, "Collision detection and reaction: A contribution to safe physical human-robot interaction," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2008, pp. 3356–3363.
- [16] S. Haddadin, A. Albu-Schaffer, A. De Luca, and G. Hirzinger, "Soft-tissue injury in robotics," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2010, pp. 3426–3433.
- [17] A. D. Dragan, S. Bauman, J. Forlizzi, and S. S. Srinivasa, "Effects of robot motion on human-robot collaboration," in *Proc. ACM/IEEE Int. Conf. Hum.-Robot Interact.*, 2015, pp. 51–58.
- [18] J. A. E. Andersson *et al.*, "CasADI - A software framework for nonlinear optimization and optimal control," *Math. Program. Comput.*, vol. 11, no. 1, pp. 1–36, 2019.
- [19] A. Wächter *et al.*, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Math. Program.*, vol. 106, no. 1, pp. 25–57, 2006.
- [20] C. Gabellieri *et al.*, "Towards an autonomous unwrapping system for intralogistics," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 4603–4610, Oct. 2019.
- [21] Z. Cao, G. Hidalgo, T. Simon, S. -E. Wei, and Y. Sheikh, "Openpose: Realtime multi-person 2d pose estimation using part affinity fields," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 1, pp. 172–186, Jan. 2021.
- [22] L. Sciacavico *et al.*, *Modelling and Control of Robot Manipulators*. Berlin, Germany: Springer Science & Business Media, 2012.
- [23] O. Khatib, "Inertial properties in robotics manipulation: An object-level framework," *Int. J. Rob. Res.*, vol. 14, no. 1, pp. 19–36, 1995.